Starter data

Create a PyCharm project:

```
pydantic_assessment
```

Install:

```
python -m pip install pydantic
```

Create:

```
registrations.json
```

Paste:

```
[
  {
    "registration_id": "R101",
    "name": "Meera Das",
    "email": "meera@example.com",
    "age": 28,
    "ticket_type": "Premium",
    "amount": 2500
  },
  {
    "registration_id": "R102",
    "name": "Joel Mathew",
    "email": "joel@example.com",
    "age": 17,
    "ticket_type": "Standard",
    "amount": 1200
  },
  {
    "registration_id": "R103",
    "name": "Kavya Rao",
    "email": "invalid-email",
    "age": 32,
    "ticket_type": "VIP",
```

```
    "amount": -500
  },
  {
    "registration_id": "R104",
    "name": "Feroz Ali",
    "email": "feroz@example.com",
    "age": 41,
    "ticket_type": "Standard",
    "amount": 1200
  }
]
```

Assessment

1. Create a Pydantic model called `Registration`.
2. Add these fields:

```
registration_id
name
email
age
ticket_type
amount
```

3. Validate that `age` must be at least `18`.
4. Validate that `amount` must be greater than `0`.
5. Restrict `ticket_type` to:

```
Standard
Premium
VIP
```

6. Load `registrations.json` and validate each record using the Pydantic model.
7. Print only the valid registrations.
8. Catch Pydantic validation errors and print a simple message showing:

- registration ID

- field with error
- error message

9. Serialize one valid Pydantic object into a Python dictionary.
10. Serialize one valid object into JSON.
11. Save all valid registrations into:

```
valid_registrations.json
```

12. Configure Python logging with these levels:

```
INFO
WARNING
ERROR
```

13. Log an `INFO` message when a registration is successfully validated.

Example expected style:

```
INFO - Registration R101 validated successfully
```

14. Log an `ERROR` message when validation fails.
15. Log a `WARNING` when the ticket type is `VIP` .
16. Save logs into:

```
registration.log
```

17. Add error handling for:

- file not found
- invalid JSON
- validation failure

18. If `registrations.json` does not exist, the program should not crash. It should print/log a meaningful error.
19. Modify one JSON record so that `age` contains text instead of a number and observe how Pydantic handles it.

20. At the end, print a summary:

```
Total records:
Valid records:
Invalid records:
```